

考虑多分类收集盒与双臂协同任务的多机械臂调度优化研究

2026 年 6 月 14 日

目录

1	问题描述	3
1.1	实际问题描述	3
1.2	集合、参数与执行模式	4
1.2.1	任务处理时间	5
1.2.2	任务处理能耗	5
1.3	决策变量	6
1.4	约束条件	7
1.4.1	任务唯一执行模式约束	7
1.4.2	时间一致性约束	7
1.4.3	机械臂资源互斥约束	7
1.4.4	序列相关空载转移约束	7
1.4.5	中心安全区互斥约束	8
1.4.6	最大完工时间、总能耗与负载均衡约束	8
1.5	目标规划模型	8
2	优化方法	9
2.1	总体求解框架	9
2.2	基础方法：CP-SAT 基准求解	9
2.2.1	求解对象	10
2.2.2	CP-SAT 的搜索机制	10
2.3	自编方法一：隐枚举精确搜索	10
2.3.1	决策过程	11
2.3.2	剪枝决策	14
2.4	自编方法二：启发式快速搜索	15

2.4.1	初始解构造	15
2.4.2	大邻域改进	16
2.5	自编方法三：热启动后隐枚举	17
2.6	二臂/三臂模式选择方法	19
2.7	速度-能耗非线性优化	20

1 问题描述

本章根据多机械臂搬运场景，对实际问题进行数学抽象。本文研究的问题不是机械臂的运动学、动力学或底层轨迹控制问题，而是将多台机械臂视为可分配的作业资源，研究在给定任务、作业空间和安全约束下，如何安排每个物块的执行机械臂、执行顺序和运行模式，使整体搬运过程在时间、能耗和负载均衡方面取得较优结果。

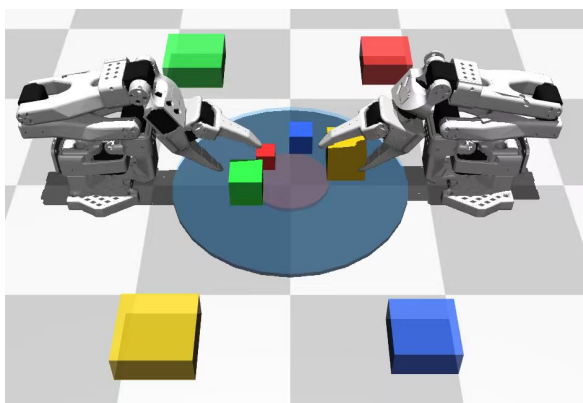
1.1 实际问题描述

考虑一个圆形作业区域，区域中随机放置若干待搬运方块。每个方块具有类型、重量、初始位置和目标收集盒。系统中可配置两只或三只机械臂，机械臂从各自的待机位置出发，将方块搬运到对应类型的收集盒中。四类方块的任务属性如下：

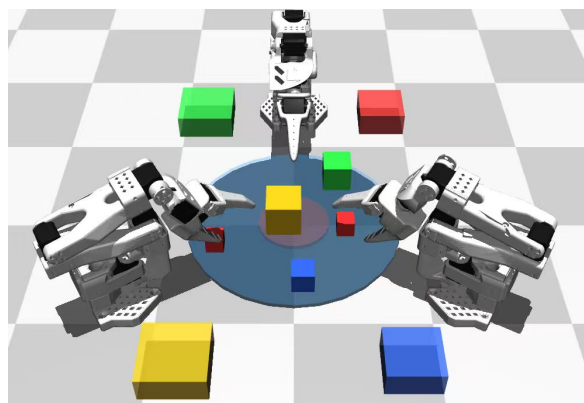
表 1: 方块类型与任务属性

方块类型	任务类型	所需机械臂数	目标收集盒	相对搬运难度
Type1	单臂任务	1	Box1	小而轻
Type2	单臂任务	1	Box2	小但较重
Type3	双臂协同任务	2	Box3	大且需协同
Type4	双臂协同任务	2	Box4	大且较重，协同代价高

图 1 给出同一组任务实例在两臂系统和三臂系统下的工作空间示意。图中蓝色圆形区域表示方块随机生成的作业区域，橙色虚线圆表示中心安全参考区，方形标记表示机械臂基座，菱形标记表示四类收集盒，圆点表示待搬运方块，虚线表示方块到对应收集盒的搬运方向。方块旁标注的机械臂组合为该案例求解得到的任务分配结果。



(a) 两臂系统示意



(b) 三臂系统示意

图 1: 多机械臂分类搬运任务的工作空间示意

在两臂系统中，双臂协同任务只能由两只机械臂共同完成；在三臂系统中，双臂协同任务可以从三只机械臂中选择任意两只共同完成。因此，三臂系统一方面增加了可选

执行模式和并行能力，可能缩短最大完工时间；另一方面也会带来额外的系统固定能耗和协同协调代价。本文后续案例分析正是围绕这一时间-能耗权衡展开。

此外，作业区域中心附近设置安全区。当某个任务的搬运路径经过中心安全区时，该任务组需要占用中心区资源。这里的中心区不是单只机械臂层面的互斥资源，而是任务组层面的共享安全资源：同一个双臂协同任务内部的两只机械臂允许同时进入中心区，但不同任务组不能同时占用中心区。

1.2 集合、参数与执行模式

设任务集合为

$$\mathcal{J} = \{1, 2, \dots, n\}.$$

两臂系统和三臂系统的机械臂集合分别为

$$\mathcal{A}^{(2)} = \{\text{arm1}, \text{arm2}\}, \quad \mathcal{A}^{(3)} = \{\text{arm1}, \text{arm2}, \text{arm3}\}.$$

下文若不特别区分两臂或三臂，统一记机械臂集合为 $\mathcal{A}$ ，其规模为 $|\mathcal{A}| = m$ ，其中 $m \in \{2, 3\}$ 。

每个任务 $j \in \mathcal{J}$ 对应一个方块。为便于后续建模，表 2 汇总主要任务参数。

表 2: 任务与系统参数说明

符号	含义
p_j	任务 j 对应方块的初始位置
q_j	任务 j 对应目标收集盒的位置
b_j	方块类型, $b_j \in \{1, 2, 3, 4\}$
r_j	完成任务 j 所需机械臂数量
w_j	方块重量
d_j	方块从初始位置到目标收集盒的搬运距离

不同方块类型决定任务所需机械臂数量：

$$r_j = \begin{cases} 1, & b_j \in \{1, 2\}, \\ 2, & b_j \in \{3, 4\}. \end{cases}$$

搬运距离定义为

$$d_j = \|p_j - q_j\|_2.$$

对每个任务 j ，根据可达性和所需机械臂数构造可行执行模式集合 $\mathcal{M}_j$ 。一个执行模式 $k \in \mathcal{M}_j$ 包含以下信息：

$$k = (j, A_k, t_k, e_k, z_k),$$

其含义如表 3 所示。

表 3: 执行模式 k 的组成

符号	含义
j	该模式对应的任务编号
A_k	执行该模式占用的机械臂集合, 满足 $A_k \subseteq \mathcal{A}$ 且 $ A_k = r_j$
t_k	该模式下的任务处理时间
e_k	该模式下的任务处理能耗
z_k	是否经过中心安全区, $z_k = 1$ 表示经过, $z_k = 0$ 表示不经过

1.2.1 任务处理时间

本文将机械臂完成一个任务的时间分为两部分：一是执行当前任务本身所需的处理时间，二是任务之间的空载转移时间。这里先定义任务处理时间；空载转移时间将在后续序列相关约束中单独建模。

任务处理时间只描述机械臂到达方块位置后，将方块搬运到目标收集盒所需的时间。它由负载搬运时间、抓取/放置服务时间和安全缓冲时间组成。设单臂负载速度为 v_s ，双臂协同负载速度为 v_d ，则

$$t_k = \begin{cases} (d_j/v_s + T_s + T_{\text{safe}}) \cdot \kappa_t, & |A_k| = 1, \\ (d_j/v_d + T_d + T_{\text{safe}}) \cdot \kappa_t, & |A_k| = 2, \end{cases}$$

其中 d_j/v_s 或 d_j/v_d 表示带负载搬运阶段的行走时间； T_s, T_d 分别表示单臂和双臂任务的抓取、夹持稳定与放置服务时间； T_{safe} 为调度层面的等效安全缓冲时间，用于表示对准、同步、放置稳定等未单独细分的短时间余量。 κ_t 为时间离散化尺度，代码实现中取 $\kappa_t = 100$ ，即将 1 秒连续时间转换为 100 个整数 tick，以便整数规划和分枝定界算法处理。

1.2.2 任务处理能耗

任务处理能耗采用等效能耗模型描述，用于在调度层面区分不同方块类型的搬运难度。该能耗并不是对电机电流、关节力矩等底层物理量的精确预测，而是根据方块重量、搬运距离、任务持续时间和双臂协同复杂度构造的综合评价指标。这里定义的处理能耗记为 e_k ，即后文系统总能耗公式中 $\sum e_k x_k$ 对应的任务执行能耗项。

对任务 j 的某个执行模式 k ，其处理能耗可表示为

$$e_k = \kappa_e \left(\phi_j E_{jk}^{\text{load}} + E_{jk}^{\text{grasp}} + |A_k| h_j \frac{t_k}{\kappa_t} + \sigma_j \eta_m \right).$$

其中， E_{jk}^{load} 为基础负载搬运能耗，与方块重量和搬运距离有关； E_{jk}^{grasp} 为抓取、夹持建立和放置动作产生的服务能耗； $|A_k| h_j t_k / \kappa_t$ 表示任务执行期间参与机械臂维持夹持与稳

定搬运所需的保持能耗； $\sigma_j \eta_m$ 表示双臂协同任务中的额外同步协调代价及系统规模修正。 κ_e 为能耗离散化尺度，用于将连续能耗转换为整数能耗值，以便与整数规划和分枝定界算法统一。

为刻画不同方块类型对能耗的影响，引入表 4 中的类型相关参数。

表 4: 方块类型相关能耗参数

参数	名称	含义
ϕ_j	载荷修正系数	放大搬运难度对能耗的影响
h_j	保持功率	单位时间内的夹持与稳定搬运代价
σ_j	同步能耗	双臂协同任务的一次性协调代价

对于 Type1 和 Type2 单臂任务，任务不涉及双臂同步，因此令 $\sigma_j = 0$ ，且 $|A_k| = 1$ 。对于 Type3 和 Type4 双臂协同任务， σ_j 取正值， $|A_k| = 2$ ，用于刻画两只机械臂在同步支撑、同步移动和协调放置过程中的额外能耗。两臂系统中 $\eta_m = 1$ ；三臂系统虽然每个双臂任务仍只使用两只机械臂，但可选组合更多、调度协调更复杂，因此 η_m 可取略大于 1 的值。其中 Type4 方块的搬运难度更高，因此其 ϕ_j 、 h_j 或 σ_j 可设置为更大的取值。

1.3 决策变量

为了同时描述任务模式选择、时间安排和机械臂作业顺序，定义如下决策变量。

- $x_k \in \{0, 1\}$: 若任务执行模式 k 被选择，则 $x_k = 1$ ，否则为 0。
- S_k, E_k : 模式 k 对应任务处理段的开始时间和结束时间。
- $y_{a,u,v} \in \{0, 1\}$: 对机械臂 a ，若其作业顺序中模式 u 后紧接模式 v ，则 $y_{a,u,v} = 1$ 。
- $C_{\max}$: 所有任务的最大完工时间。
- E_{tot} : 系统总能耗。
- L_a : 机械臂 a 的总负载时间，包括任务处理时间和空载转移时间。
- B : 机械臂负载不均衡度，即最大负载与最小负载之差。

其中， $y_{a,u,v}$ 用于描述序列相关 setup。机械臂执行完一个任务后，需要从上一任务目标收集盒移动到下一任务的方块初始位置；若该任务是该机械臂的第一个任务，则需要从机械臂待机位置移动到该方块位置。这部分空载转移时间和能耗都与任务顺序有关。

1.4 约束条件

1.4.1 任务唯一执行模式约束

每个任务必须且只能选择一种可行执行模式：

$$\sum_{k \in \mathcal{M}_j} x_k = 1, \quad \forall j \in \mathcal{J}.$$

该约束保证每个方块都被搬运一次，且不会被多个模式重复执行。

1.4.2 时间一致性约束

若模式 k 被选择，则其结束时间等于开始时间加处理时间：

$$E_k = S_k + t_k, \quad \forall k \text{ with } x_k = 1.$$

在整数规划实现中，未被选择的模式通过可选区间变量处理；在自编算法中，未被选择的模式不会进入调度序列。

1.4.3 机械臂资源互斥约束

同一只机械臂在同一时刻不能执行两个任务。若两个被选择模式 u, v 均占用机械臂 a ，则必须满足二者在时间轴上不重叠：

$$[S_u, E_u) \cap [S_v, E_v) = \emptyset, \quad \forall a \in A_u \cap A_v.$$

基础模型中该约束通过 NoOverlap 资源约束实现；自编求解器中则通过维护每只机械臂的可用时间递推生成可行调度。

1.4.4 序列相关空载转移约束

设机械臂 a 执行完模式 u 后紧接执行模式 v ，则模式 v 的开始时间必须不早于模式 u 的结束时间加空载转移时间：

$$S_v \geq E_u + \tau_{u,v}, \quad \text{if } y_{a,u,v} = 1.$$

其中

$$\tau_{u,v} = \frac{\|q_u - p_v\|_2}{v_0} \cdot \kappa_t,$$

v_0 为空载移动速度。若 v 是机械臂 a 的第一个任务，则

$$S_v \geq \frac{\|o_a - p_v\|_2}{v_0} \cdot \kappa_t,$$

其中 o_a 为机械臂 a 的待机位置。

相应的空载转移能耗记为

$$\epsilon_{u,v} = \alpha_0 \|q_u - p_v\|_2 \cdot \kappa_e,$$

首任务从待机位置出发时同理计算。该项将计入系统总能耗。

1.4.5 中心安全区互斥约束

设 $\mathcal{C}$ 为经过中心安全区的已选任务模式集合。对于任意两个不同任务组 $u, v \in \mathcal{C}$ ，其中心区占用时间不能重叠：

$$[S_u, E_u) \cap [S_v, E_v) = \emptyset, \quad u \neq v.$$

该约束体现中心安全区在任务组层面的互斥。需要强调的是，对于同一个双臂协同任务，其内部两只机械臂属于同一任务组，不构成彼此冲突。

1.4.6 最大完工时间、总能耗与负载均衡约束

最大完工时间满足

$$C_{\max} \geq E_k, \quad \forall k \text{ with } x_k = 1.$$

系统总能耗由处理能耗、空载转移能耗和系统级固定开销组成：

$$E_{\text{tot}} = \sum_{j \in \mathcal{J}} \sum_{k \in \mathcal{M}_j} e_k x_k + \sum_{a \in \mathcal{A}} \sum_{(u,v)} \epsilon_{u,v} y_{a,u,v} + m E_{\text{sys}},$$

其中 E_{sys} 为单只机械臂的启动、上电、伺服保持和控制通信等效固定能耗。

机械臂负载 L_a 包括该机械臂承担的处理时间和空载转移时间。负载均衡指标定义为

$$B = \max_{a \in \mathcal{A}} L_a - \min_{a \in \mathcal{A}} L_a.$$

1.5 目标规划模型

本问题同时关注完成效率、运行能耗和机械臂利用均衡。由于三个目标的重要性不同，本文不采用简单加权求和，而采用序贯多目标优化思想：先优化最高优先级目标，再在保持高优先级目标最优的前提下优化下一目标。

第一阶段以最大完工时间最小为目标：

$$P_1 : \quad \min C_{\max}.$$

记第一阶段得到的最优值为 $C_{\max}^*$ 。第二阶段在保持最优完工时间不变的条件下，最小化系统总能耗：

$$P_2 : \quad \min E_{\text{tot}}, \quad \text{s.t. } C_{\max} = C_{\max}^*.$$

记第二阶段得到的最优能耗为 E_{tot}^* 。第三阶段在保持前两级目标最优的条件下，最小化机械臂负载不均衡度：

$$P_3 : \quad \min B, \quad \text{s.t. } C_{\max} = C_{\max}^*, \quad E_{\text{tot}} = E_{\text{tot}}^*.$$

该目标结构表示：首先保证整体任务尽快完成，其次在不牺牲时间目标的前提下降低能耗，最后在前两者基础上改善机械臂负载均衡。这样处理避免了人为选择加权系数，也使求解结果的优先级解释更加明确。

综上，本文的核心模型可以理解为一个带有可选执行模式、并行机器资源约束、序列相关 setup、中心安全区互斥和多目标优先级的调度优化模型。后续第 3 章将在该模型基础上分别设计基础求解方法、自编优化算法以及速度-能耗层的非线性扩展。

2 优化方法

第 2 章已经给出了多机械臂分类搬运问题的调度模型。本章重点说明模型如何被求解，即不同算法在每一步如何做决策、如何排除无效方案，以及不同方法之间的逻辑差异。整体上，本文先求解离散调度问题，得到任务分配和作业顺序；随后再基于调度结果进行二臂/三臂模式选择和速度-能耗扩展分析。

2.1 总体求解框架

本文的优化流程如图 2 所示。输入任务实例后，首先构造每个任务的可行执行方式；随后分别采用 CP-SAT 基准方法、自编隐枚举精确搜索、启发式快速搜索和热启动后隐枚举进行调度求解；最后对不同系统配置和速度策略进行分析。

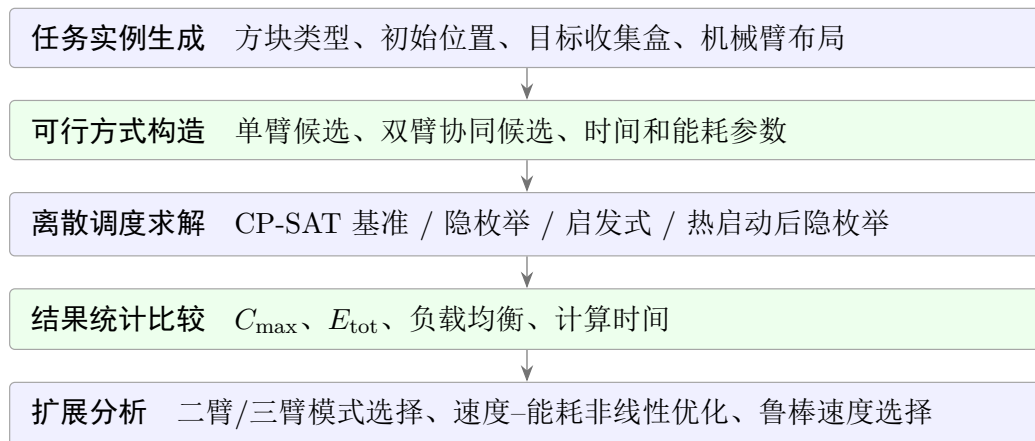


图 2: 本文优化方法总体框架

上述方法的分工如下：CP-SAT 提供通用求解器基准；隐枚举用于给出可解释的精确搜索过程；启发式用于快速获得较好可行解；热启动后隐枚举则把启发式的快速上界和隐枚举的最优性证明结合起来。

2.2 基础方法：CP-SAT 基准求解

基础方法将第 2 章的调度模型交给 OR-Tools CP-SAT 求解。它的作用是提供可靠基准，而不是展示人工设计的搜索规则。CP-SAT 适合处理本问题，是因为它可以同时

处理 0-1 选择、整数时间变量、逻辑条件和资源不重叠关系。

2.2.1 求解对象

为说明 CP-SAT 接收到的对象，仍沿用第 2 章的变量： x_k 表示执行模式 k 是否被选择， $[S_k, E_k)$ 表示该模式对应的作业区间， $y_{a,u,v}$ 表示机械臂 a 是否在模式 u 后紧接执行模式 v 。也就是说，CP-SAT 需要同时决定模式选择、作业区间位置和同一机械臂上的前后衔接关系。

在本问题中，CP-SAT 接收到的不是一条已经排好的任务序列，而是一组尚未确定的逻辑变量和整数时间变量。求解器需要同时决定三类内容：每个任务选择哪种执行方式、每个被选方式何时开始和结束、同一机械臂上的任务按什么顺序衔接。

2.2.2 CP-SAT 的搜索机制

CP-SAT 的求解过程可以理解为“约束传播 + 分支搜索 + 冲突学习 + 目标界更新”的循环。

首先，求解器为每个变量维护一个可能取值范围。例如， x_k 的取值范围为 $\{0,1\}$ ，开始时间 S_k 的取值范围为某个整数时间窗。约束传播会在尚未完全确定解之前不断缩小这些范围。若某个任务只剩一个候选方式没有被排除，唯一选择关系会直接推出该方式必须被选择；若某只机械臂上两个已存在作业的时间窗无法同时容纳，不重叠关系会迫使其中一个作业推迟，或判定当前分支不可行。

当传播无法继续推进时，求解器进入分支搜索。它选择一个尚未确定的逻辑决策或整数变量进行试探，例如假设某个候选方式被选择，或假设两个作业的前后顺序为 u 在 v 之前。每做出一个试探，相关约束又会触发新的传播；若出现矛盾，则说明当前试探组合不能同时成立。

出现冲突后，CP-SAT 不只是简单回退，而会进行冲突分析，提取导致矛盾的一组关键决策，并形成一条避免重复进入类似矛盾的逻辑信息。对于优化问题，求解器还会维护当前最好目标值和可能达到的下界；一旦某个分支已经不可能优于当前最好解，就可以被剪去。本文的序贯目标也按这一机制实现：每一阶段求得最优值后，将其作为下一阶段的附加条件。

因此，CP-SAT 的优势是通用、稳定、自动化程度高；不足是其剪枝主要发生在求解器内部，不容易直接解释为本问题中的某个调度结构。后续自编方法则反过来，显式维护部分调度状态，并由程序主动决定下一步搜索方向。

2.3 自编方法一：隐枚举精确搜索

第一类自编方法是隐枚举精确搜索。它不依赖通用求解器，而是由程序逐步构造调度序列。搜索目标采用字典序

$$(C_{\max}, E_{\text{tot}}, B),$$

即先比较完工时间，再比较能耗，最后比较负载均衡。

2.3.1 决策过程

隐枚举每一步要回答的问题是：在当前部分调度之后，下一个安排哪个任务，以及采用哪一种执行方式。程序从空调度开始，逐步扩展调度序列。每扩展一步时，它枚举“未完成任务-可行执行方式”的组合，并计算该组合加入当前调度后对应的开始时间、结束时间、转移代价和目标值变化。

对一个候选决策的评价分为两层。本章用 Ω 表示一个部分调度状态； $C_{\text{cur}}(\Omega)$ 表示该状态下已安排任务的当前最大完成时刻； $\theta_a(\Omega)$ 表示机械臂 a 在该状态下的可用时间， $\xi_a(\Omega)$ 表示机械臂 a 的当前位置， $\theta_{\text{center}}(\Omega)$ 表示中心区在该状态下的最早释放时间。这些符号只用于描述算法状态，不改变第 2 章模型变量。

第一层是直接评价，即把任务 j 的某个执行模式 $k \in \mathcal{M}_j$ 暂时接到当前部分调度 Ω 后，计算它真实插入后的时间和能耗。候选中机械臂 a 的准备完成时间为

$$R_{ak}(\Omega) = \theta_a(\Omega) + \tau(\xi_a(\Omega), p_j), \quad a \in A_k.$$

双臂任务必须等所有参与机械臂到齐，因此未考虑中心区时的最早同步开始时间为

$$\tilde{S}_k(\Omega) = \max_{a \in A_k} R_{ak}(\Omega).$$

若该任务需要占用中心区，则开始时间继续向后推迟到不与已有中心区区间冲突的位置，记为

$$S_k(\Omega) = \text{NextFree}_{\text{center}}(\tilde{S}_k(\Omega), t_k), \quad E_k(\Omega) = S_k(\Omega) + t_k.$$

若不占用中心区，则 $S_k(\Omega) = \tilde{S}_k(\Omega)$ 。由此可以得到该候选的 setup 时间、中心区等待、结束时间、增量能耗以及执行后的机械臂位置。

第二层是前瞻评价。这里的“理想完成”不是构造一条真实调度，而是把剩余问题放松，计算任何真实后续调度都不可能突破的下界。设当前候选加入后的部分调度状态为 Ω ，剩余任务集合为 $\mathcal{R}(\Omega) \subseteq \mathcal{J}$ ，当前最好完整解为

$$Z^{\text{best}} = (C_{\text{max}}^{\text{best}}, E_{\text{tot}}^{\text{best}}, B^{\text{best}}).$$

算法为该候选计算一个乐观目标

$$\underline{Z}(\Omega) = (\underline{C}(\Omega), \underline{E}(\Omega), 0),$$

其中负载均衡项先取 0，表示在前两级目标已经无法改进时才需要继续比较均衡性。若 $\underline{Z}(\Omega)$ 按字典序已经不优于 Z^{best} ，则该候选之后无论怎样安排都不可能得到更好的完整解，可以直接丢弃。

完工时间下界 $\underline{C}(\Omega)$ 由多个放松下界取最大值：

$$\underline{C}(\Omega) = \max\{\underline{C}_{\text{task}}, \underline{C}_{\text{cap}}, \underline{C}_{\text{arm}}, \underline{C}_{\text{center}}\}.$$

各项的计算方式如下：

$$\begin{aligned}
 \underline{C}_{\text{task}} &= \max \left\{ C_{\text{cur}}(\Omega), \max_{j \in \mathcal{R}(\Omega)} \min_{k \in \mathcal{M}_j} E_k(\Omega) \right\}, \\
 W_{\text{rem}}(\Omega) &= \sum_{j \in \mathcal{R}(\Omega)} \min_{k \in \mathcal{M}_j} (t_k |A_k| + \rho W_{jk}^{\text{setup}}(\Omega)), \\
 \underline{C}_{\text{cap}} &= \max \left\{ C_{\text{cur}}(\Omega), \left\lceil \frac{\sum_{a \in \mathcal{A}} \theta_a(\Omega) + W_{\text{rem}}(\Omega)}{|\mathcal{A}|} \right\rceil \right\}, \\
 \underline{C}_{\text{arm}} &= \max_{a \in \mathcal{A}} \left\{ \theta_a(\Omega) + \sum_{\substack{j \in \mathcal{R}(\Omega) \\ a \in \cap_{k \in \mathcal{M}_j} A_k}} \min_{k \in \mathcal{M}_j} t_k \right\}, \\
 \underline{C}_{\text{center}} &= \max \left\{ C_{\text{cur}}(\Omega), \theta_{\text{center}}(\Omega) + \sum_{j \in \mathcal{R}(\Omega)} \min_{k \in \mathcal{M}_j} z_k t_k \right\}.
 \end{aligned}$$

其中 $W_{jk}^{\text{setup}}(\Omega)$ 为任务 j 在模式 k 下的最小可能 setup 工作量， $\rho \in \{0, 1\}$ 表示是否启用 setup 容量下界。设

$$\mathcal{P}(\Omega) = \{\xi_a(\Omega) : a \in \mathcal{A}\} \cup \{\text{各收集盒位置}\},$$

则可取

$$W_{jk}^{\text{setup}}(\Omega) = |A_k| \cdot \min_{p \in \mathcal{P}(\Omega)} \left(\frac{\|p - p_j\|_2}{v_0} \cdot \kappa_t \right).$$

这表示：未来某个剩余任务开始前，参与机械臂至少要从当前所在位置或某个已完成任务的收集盒位置移动到该任务起点。由于公式取的是所有可能出发点中距离最短的一个，所以它只会低估真实 setup，不会高估。代码中 ρ 采用自适应设置：默认取 0；当启用 setup 容量下界开关，或在二臂系统且 $|\mathcal{R}(\Omega)| \geq 4$ 时，取 $\rho = 1$ 。这样设置的原因是，setup 容量下界需要为每个剩余任务估计最小空载转移工作量，计算量比只统计处理时间更高；若剩余任务很少，直接继续分支通常更快，额外计算下界未必能带来明显剪枝收益。相反，在二臂系统中，机械臂资源更紧，空载转移会更直接地影响完工时间；当剩余任务不少于 4 个时，后续分支数量开始明显增长，此时把最小 setup 工作量加入容量下界，往往能提前排除一批实际不可能优于当前解的候选。

上述四个下界分别对应四种“无论后续怎么排都绕不开”的限制：

- $\underline{C}_{\text{task}}$ 是单任务最早完成下界。 $C_{\text{cur}}(\Omega)$ 是当前部分调度已经达到的完成时间，后续安排不可能把它变小；而 $E_k(\Omega)$ 表示在当前状态下把某个剩余任务作为下一任务执行时的完成时刻。这里要与 $C_{\text{cur}}(\Omega)$ 取最大值，是因为某个剩余任务可能由空闲机械臂很快完成，其 $E_k(\Omega)$ 反而小于当前已经形成的最大完成时刻。
- $\underline{C}_{\text{cap}}$ 是机械臂总容量下界。它不把所有剩余任务串行相加，而是把剩余处理工作量按机械臂数量平均分摊；双臂任务按 $t_k |A_k|$ 计入，是因为它会同时占用两只机械臂。若启用 setup 下界，则额外加入最小可能空载转移工作量。

- $\underline{C}_{\text{arm}}$ 是强制机械臂下界。如果某些剩余任务的所有可行模式都必须包含同一只机械臂 a ，这些任务无论如何都要排到该机械臂后面，因此要从 $\theta_a(\Omega)$ 继续累加其最短处理时间。
- $\underline{C}_{\text{center}}$ 是中心区容量下界。中心区是单容量共享资源，同一时刻只能有一个任务组占用；因此对不可避免的中心区占用时间做串行累计。若某任务存在不经过中心区的可行模式，则 $\min_{k \in \mathcal{M}_j} z_k t_k$ 可以取到 0。

能耗下界取

$$\underline{E}(\Omega) = E_{\text{done}}(\Omega) + \sum_{j \in \mathcal{R}(\Omega)} \min_{k \in \mathcal{M}_j} e_k + mE_{\text{sys}},$$

即当前已发生能耗加上每个剩余任务可行模式中的最小处理能耗，再加第 2 章总能耗模型中的系统固定开销 mE_{sys} 。这里的 $E_{\text{done}}(\Omega)$ 不只是已完成任务的处理能耗，还包括已经确定顺序的空载转移能耗：

$$E_{\text{done}}(\Omega) = \sum_{\text{已选 } k} e_k + \sum_{\text{已确定转移 } (u,v)} \epsilon_{u,v}.$$

因此，已经发生的 setup 能耗是计入总能耗的。对于尚未安排的剩余任务，未来空载转移路径还没有确定；若强行加入某个具体 setup 能耗，可能会高估下界并误剪最优解。因此能耗下界中暂不加入未来 setup 能耗，只保留剩余任务最小处理能耗。相应地，未来 setup 在时间下界中只通过 $W_{jk}^{\text{setup}}(\Omega)$ 以最小可能值保守加入。

这个过程如图 3 所示。



图 3: 自编隐枚举精确搜索的决策流程

候选决策不是任意顺序展开，而是按如下排序键展开：

$$\text{key}(\Omega, k) = (C_{\text{cur}}(\Omega'), E_k(\Omega), W_{\text{center}}, E_{\text{tot}}(\Omega'), B(\Omega'), W_k^{\text{setup}}, \Theta_{A_k}(\Omega), j, k),$$

其中 Ω' 为加入候选 k 后的新状态， W_{center} 为中心区等待时间， W_k^{setup} 为该候选的空载转移时间， $\Theta_{A_k}(\Omega) = \max_{a \in A_k} \theta_a(\Omega)$ 。排序只改变搜索顺序，不删除候选方案，因此不影响精确性；它的作用是更早得到较好的完整调度，从而形成更强的当前最好解。

在最终实现中，候选评价还加入了自适应机制：

$$\text{candidate kept} = \begin{cases} \text{true}, & Z^{\text{best}} \text{ 尚不存在,} \\ (C_{\max}(\Omega'), \underline{E}(\Omega'), 0) <_{\text{lex}} Z^{\text{best}}, & \text{轻量筛选阶段,} \\ \underline{Z}(\Omega') <_{\text{lex}} Z^{\text{best}}, & \text{强化筛选阶段.} \end{cases}$$

这里的自适应不是连续调参，而是按问题规模和搜索状态切换。若当前还没有任何完整可行解，即 Z^{best} 尚不存在，算法不会进行下界筛选，因为没有可比较的上界；此时只按排序键选择优先搜索顺序。若已经得到一个完整解，则候选阶段即开始使用下界与 Z^{best} 比较。需要注意的是，小规模算例并非完全不使用能耗下界：代码仍会计算 $\underline{E}(\Omega')$ ，即当前已发生能耗、剩余任务最小处理能耗和系统固定开销之和；只是时间下界只取插入该候选后的当前完工时间 $C_{\max}(\Omega')$ ，不额外计算机械臂容量、强制机械臂、中心区容量等较重的时间下界。当任务数量达到较大规模时，才启用由强化时间下界和能耗下界共同组成的强化筛选。本文实现中以任务数量为主要阈值，当 $n \geq 8$ 且已有当前最好解时启用强化候选筛选；其中 setup 容量下界进一步要求二臂系统且剩余任务数 $|\mathcal{R}(\Omega)| \geq 4$ ，或手动打开 setup 容量下界开关。这一附加条件对应的是收益与开销的平衡：三臂系统的空闲缓冲更大，单个 setup 下界对容量瓶颈的刻画不如二臂系统敏感；而剩余任务少于 4 个时，搜索树已经接近叶节点，直接枚举通常比额外计算 setup 容量下界更划算。这样小规模算例仍保留便宜的能耗筛选，大规模算例又能在搜索树较高层剪去大量无希望分支。

通过候选阶段筛选的分支会按排序键依次展开。也就是说，算法先把某个候选模式 k 加入当前部分调度，得到子状态 Ω' ；随后递归进入 Ω' ，并在该子状态入口处进行正式的支配剪枝和下界剪枝。若子状态没有被剪去，算法才继续从 Ω' 生成下一层候选。

候选筛选和节点剪枝并不是完全重复。候选筛选发生在“是否进入这个下一步选择”之前，主要用于提前丢弃明显不可能改进当前最好解的候选；而节点剪枝发生在“已经进入某个部分调度状态”之后，判断这个状态是否还值得继续扩展。保留节点剪枝有三点原因：第一，小规模候选阶段只使用轻量时间下界 $C_{\max}(\Omega')$ ，而节点剪枝会重新计算由单任务最早完工、机械臂容量、强制机械臂和中心区容量组成的完整时间下界组合，并配合同一类能耗下界 $\underline{E}(\Omega')$ ；第二，节点剪枝还包含状态支配判断，这不是候选排序键能替代的；第三，候选列表生成后，前面展开的分支可能找到更好的完整解并更新 Z^{best} ，后续候选虽然在旧上界下通过了筛选，但递归进入子状态时会用更新后的上界再次判断，从而可能被正式剪去。

2.3.2 剪枝决策

隐枚举若完全展开，会面对巨大的组合空间。因此每到一个部分调度，算法都会先判断：在上述放松下界下，这个分支是否仍有可能超过当前最好解。如果答案是否定的，就停止扩展该分支。

本文使用两类正式剪枝。第一类是下界剪枝：利用剩余任务单独最早完工、机械臂容量、强制机械臂、中心区串行占用和最小剩余能耗，估计该分支能达到的最好可能结果。如果该估计已经不优于当前最好解，则剪去。第二类是状态支配剪枝：如果两个部分调度完成的任务相同、机械臂当前位置相同，而其中一个在时间、能耗、负载和中心区释放情况上都不差于另一个，则较差状态没有继续保留的必要。候选生成阶段的提前筛选可以看作正式剪枝前的一道快速过滤，它减少进入递归的候选数量；节点剪枝则是搜索树上的保守判定，保证每个真正进入的部分调度状态仍然满足继续扩展的必要性。

隐枚举方法的优点是结果可证明最优，而且每一次搜索和剪枝都有明确含义；不足是任务规模较大时，即使有剪枝，也可能需要较长时间。

2.4 自编方法二：启发式快速搜索

第二类方法是启发式快速搜索。它不追求证明全局最优，而是尽快得到质量较好的可行调度，适合任务数量较多或需要批量实验的场景。

2.4.1 初始解构造

启发式首先从多个角度构造初始调度。不同策略优先考虑不同因素，例如尽快完成任务、减少中心区等待、降低能耗或改善机械臂负载均衡。这样做是为了避免单一贪心规则过早固定某一种任务顺序。

在构造过程中，算法每次从未安排任务中挑选一个候选任务及其执行方式，并立即把它插入当前调度。局部评分采用简化前瞻形式：

$$H(\Omega') = (\hat{C}(\Omega'), C_{\text{cur}}(\Omega'), E_{\text{done}}(\Omega') + \underline{E}_{\text{rem}}(\Omega'), B(\Omega')),$$

其中 Ω' 为插入后的状态。这里的“简化前瞻”可以理解为：不完整求解剩余调度，只用很便宜的乐观估计判断这个插入动作是否会给后续留下过重负担。具体地，令

$$D_{\text{rem}}(\Omega') = \sum_{j \in \mathcal{R}(\Omega')} \min_{k \in \mathcal{M}_j} t_k, \quad \underline{E}_{\text{rem}}(\Omega') = \sum_{j \in \mathcal{R}(\Omega')} \min_{k \in \mathcal{M}_j} e_k.$$

其中 $D_{\text{rem}}(\Omega')$ 表示剩余任务至少还需要的处理时间总量， $\underline{E}_{\text{rem}}(\Omega')$ 表示剩余任务至少还会产生的处理能耗。算法进一步把剩余处理时间按机械臂数量平均分摊，得到乐观完工估计

$$\hat{C}(\Omega') = \max \left\{ C_{\text{cur}}(\Omega'), \left\lceil \frac{\sum_{a \in \mathcal{A}} \theta_a(\Omega') + D_{\text{rem}}(\Omega')}{|\mathcal{A}|} \right\rceil \right\}.$$

因此， $H(\Omega')$ 的含义是按顺序比较四件事：该插入后从平均负载角度看未来最早可能多晚完成、当前已经形成的最大完成时间、当前能耗加剩余最小能耗、以及机械臂负载差。该评分不是用于证明最优的严格下界，而是用于排序候选决策；它比只看当前任务结束时间更稳，因为它会惩罚那些当前完成很快、但把大量剩余工作集中留给后续的选择。多个初始解生成后，算法保留字典序目标最好的一个作为后续改进起点。

为了进一步提高初始解质量，启发式中还加入了限宽 beam search。它不是只保留一条当前最优的部分调度，而是在每一层保留若干条评分靠前的部分调度路径；下一层再从这些路径继续扩展。beam 中部分调度的排序键为

$$(\hat{C}(\Omega), C_{\text{cur}}(\Omega), E_{\text{done}}(\Omega) + \underline{E}_{\text{rem}}(\Omega), B(\Omega), |\mathcal{R}(\Omega)|),$$

其中 $\hat{C}(\Omega)$ 用当前各机械臂可用时间和剩余最短处理时间估计未来平均负载后的乐观完工时间， $\underline{E}_{\text{rem}}(\Omega)$ 是剩余任务最小能耗之和。该排序使 beam search 保留的不是“当前看起来最早”的单一路径，而是兼顾未来工作量、能耗和负载均衡的多条路径。beam 宽度根据任务数量和机械臂数量自适应调整：规模较小时保留更多路径以提高解质量，规模较大或三臂组合较多时适当收窄宽度以控制计算时间。这样比单一路径贪心更稳，又比完整枚举快得多。

2.4.2 大邻域改进

得到初始解后，算法进入大邻域搜索。一次迭代包括“破坏”和“修复”两个动作。破坏阶段从当前完整调度中移除一部分任务，被移除的任务可以随机选取，也可以优先选择造成等待、结束较晚或涉及双臂协同的任务。

修复阶段采用逐步最优插入。设破坏后留下的任务序列为基准序列，被移除任务集合为 $\mathcal{D}$ 。修复的目标不是把 $\mathcal{D}$ 中的任务放回原来的位置，而是在当前局部邻域内重新决定这些任务的执行方式和相对位置。算法每次只固定一个插入决策，具体过程为：

1. 对每个尚未插回的任务 $j \in \mathcal{D}$ ，枚举其所有可行模式 $k \in \mathcal{M}_j$ ；
2. 对每个模式，枚举它可以插入基准序列的所有位置，包括最前端、任意两个任务之间以及最后端；
3. 对每一种插入方案，将新的任务序列从头顺序解码，得到完整调度，并计算其 $(C_{\text{max}}, E_{\text{tot}}, B)$ ；
4. 选择字典序目标最小的插入方案；若 $(C_{\text{max}}, E_{\text{tot}}, B)$ 相同，则继续用中心区等待时间、setup 时间和任务编号等规则打破并列；
5. 将该任务固定在选定位置，并对剩余被移除任务重复上述过程，直到 $\mathcal{D}$ 为空。

这里需要重新解码整条序列，是因为一个任务插入到某个位置后，插入点之后的任务开始时间、机械臂当前位置、setup 时间和中心区等待都可能连锁变化，不能只评价被插入任务本身。但这种重算不会使算法退化为完整枚举，原因有三点。第一，枚举只发生在被破坏的小邻域内。设任务总数为 n ，代码中每轮破坏的基准规模为

$$q_0 = \min \left\{ \max \left(2, \left\lfloor \frac{n}{3} \right\rfloor \right), n - 1 \right\},$$

实际迭代时再在 $q_0 - 1, q_0, q_0 + 1$ 附近小幅变化, 并始终限制在 $[1, n - 1]$ 内, 因此不会把全部任务重新排列。第二, 修复采用逐个插回的贪心策略: 一次插入决策的候选数量大致为

$$|\mathcal{D}_{\text{rem}}| \times \text{平均可行模式数} \times (\text{当前序列长度} + 1),$$

选出当前最好的插入后立即固定, 再处理下一个任务; 算法并不会同时枚举 $\mathcal{D}$ 中所有任务的排列、模式组合和位置组合。第三, 大邻域搜索本身还受到时间限制、最大迭代次数和连续未改进次数限制。因此, 该步骤本质上是“局部枚举 + 贪心修复”, 用有限计算换取较好的邻域改进质量, 而不是全局完整枚举。

这种做法比简单交换两个相邻任务更灵活。多机械臂调度中, 一个任务位置改变可能同时影响多只机械臂的后续可用时间; 双臂协同任务尤其明显, 一次改变会联动两只机械臂。大邻域搜索通过一次移除多个任务, 给算法留下更大的重排空间, 更容易跳出局部最优。

新调度生成后, 算法按字典序目标判断是否接受。若新解更好, 则直接接受; 在搜索前期, 也允许少量不明显更差的解进入下一轮, 以避免过早停在局部最优; 搜索后期则更偏向稳定改进, 并尝试在不增加最大完工时间的前提下降低能耗。

大邻域搜索结束后, 算法还会进行一轮末端能耗精化。由于本文目标是序贯的, 最大完工时间优先级最高, 因此该阶段只接受不增大 $C_{\max}$ 的局部调整。具体而言, 算法依次尝试三类小改动: 为单个任务更换可行模式、将单个任务移动到其他位置、交换两个任务的位置。若某个改动保持 $C_{\max}$ 不变或更小, 并且能降低能耗或负载差, 则接受该改动并继续搜索。这个阶段不再追求大范围重排, 而是在已有较好调度附近压低次级目标。

启发式的时间分配也采用自适应策略。总时间被分给 beam search、大邻域搜索和末端能耗精化三个阶段: 任务较少时, 候选空间相对有限, 算法只给 beam search 和大邻域搜索较短时间, 并保留少量时间做末端能耗精化; 任务较多时, 更多时间分配给 beam search 和大邻域搜索, 以便先找到结构较好的任务顺序, 再在末端对能耗进行小范围修正。大邻域搜索中的破坏规模采用上述 q_0 规则, 核心含义是每轮大约重排三分之一的任务, 但至少移除 2 个、至多保留 1 个任务不被移除, 并在相邻迭代中做小幅扰动。这样每次重排既能改变足够多的任务, 又不会让修复过程变得过慢。这个自适应组合是启发式效率提升的关键: 它避免了“小问题上搜索策略过重、大问题上探索不足”的两种低效情况。

2.5 自编方法三: 热启动后隐枚举

第三类方法将前两类方法结合起来: 先用启发式快速得到一个完整可行调度, 再把它作为隐枚举精确搜索的初始上界。本文称其为热启动后隐枚举。它的核心思想不是用启发式结果替代精确搜索, 而是让精确搜索一开始就拥有一个较强的当前最好解, 从而更早触发下界剪枝。

热启动阶段本身采用自适应策略。设任务总数为 n ，机械臂数量为 m 。当

$$n \geq 8, \quad m \geq 3$$

时，代码采用完整启发式热启动；否则采用轻量热启动。两种路径的共同点是：启发式只负责给出一个完整可行调度 S^{init} ，随后隐枚举仍从空调度开始搜索，并把 S^{init} 的目标值作为初始上界

$$Z_0^{\text{best}} = (C_{\max}(S^{\text{init}}), E_{\text{tot}}(S^{\text{init}}), B(S^{\text{init}})).$$

因此，热启动不会改变可行域、候选生成方式、下界剪枝规则或状态支配规则；它改变的是隐枚举一开始用于比较的 incumbent。

轻量热启动用于小规模算例或二臂算例，其流程为

$$\begin{aligned} & \text{四个贪心完整解} \rightarrow S^{\text{greedy}} \rightarrow \text{限时0.20 s 的 beam search} \rightarrow S^{\text{beam}}, \\ & S^{\text{init}} = \arg \min_{\text{lex}} \{S^{\text{greedy}}, S^{\text{beam}}\}. \end{aligned}$$

其中 S^{greedy} 是四种贪心策略中目标值最好的完整调度； S^{beam} 是 beam search 从空调度独立构造出的完整调度，而不是在贪心解上继续修改。若 beam search 在时间限制内没有返回完整解，则直接使用 S^{greedy} 。这一路径的目的不是深度改进，而是用很小的预处理时间获得一个不差的初始上界。

完整启发式热启动用于 $n \geq 8, m \geq 3$ 的场景，其流程为

$$\begin{aligned} & \text{beam search 预热} \rightarrow S^{\text{beam}}, \\ & \text{四个贪心完整解} \rightarrow S^{\text{greedy}}, \\ & \min_{\text{lex}} \{S^{\text{beam}}, S^{\text{greedy}}\} \rightarrow \text{多启动大邻域搜索} \rightarrow \text{同 } C_{\max} \text{ 能耗精化} \rightarrow S^{\text{init}}. \end{aligned}$$

这里的 beam search 和四个贪心解同样是两个候选来源，代码先分别得到可行调度，再按字典序目标取更优者作为大邻域搜索的起点。随后 LNS 通过破坏-修复在局部邻域中改进任务顺序和执行模式；末端能耗精化只接受不增大 $C_{\max}$ 的调整，用于压低能耗或负载差。该路径只在大规模三臂场景启用，是因为此时机械臂组合更多、搜索树更大，较好的初始上界更容易转化为后续隐枚举的剪枝收益。

获得 S^{init} 后，隐枚举的执行逻辑保持为：

1. 将 Z_0^{best} 作为搜索开始时的当前最好目标值；
2. 从空调度出发，按前述候选生成、候选筛选、节点剪枝和状态支配规则完整搜索；
3. 搜索过程中若发现更好的完整调度，则更新 Z^{best} ，后续分支使用更强上界继续剪枝；
4. 若隐枚举搜索完成，则最终结果仍由分枝定界证明最优。

该方法的亮点有三点。第一，它保留了精确算法的最优性证明：启发式解只作为初始 incumbent，不会改变任何可行解，也不会改变安全下界。第二，它把启发式的优势转化为剪枝优势：初始上界越好，越多分支会因为下界不可能优于 Z^{best} 而被提前剪去。第三，它根据问题规模选择 warm start 强度，避免“小问题上预处理过重、大问题上初始上界过弱”的情况。因此，热启动后隐枚举兼具启发式的快速找解能力和隐枚举的全局最优性证明能力。

2.6 二臂/三臂模式选择方法

二臂和三臂系统的差异，首先体现在可行模式集合 $\mathcal{M}_j$ 的构造上。对每个任务 j ，代码并不是在搜索过程中临时判断“哪只机械臂能做”，而是在调度前先生成所有可行执行模式：

$$k = (j, A_k, t_k, e_k, z_k) \in \mathcal{M}_j.$$

其中 A_k 是该模式占用的机械臂集合， t_k 、 e_k 和 z_k 分别为该模式的处理时间、处理能耗和中心区标记。

具体来说，Type1 和 Type2 为单臂任务，代码会检查每只机械臂是否同时能够到达任务起点 p_j 和目标收集盒 q_j ；若机械臂 a 可达，则生成一个单臂模式

$$A_k = \{a\}.$$

Type3 和 Type4 为双臂协同任务，代码会枚举所有大小为 2 的机械臂组合，并保留组合中两只机械臂都能到达 p_j 和 q_j 的模式：

$$A_k = \{a, b\}, \quad a \neq b.$$

因此，在二臂系统中，双臂任务最多只有一个协同组合 $\{1, 2\}$ ；在三臂系统中，同一个双臂任务可能产生三个候选组合 $\{1, 2\}$ 、 $\{1, 3\}$ 和 $\{2, 3\}$ 。这意味着三臂系统不仅要决定任务顺序，还要决定每个双臂任务由哪一对机械臂完成。

在隐枚举和启发式搜索中，模式选择直接进入候选决策层。算法遍历所有尚未完成任务 $j \in \mathcal{R}(\Omega)$ 及其所有模式 $k \in \mathcal{M}_j$ ，因此“选择任务”和“选择机械臂组合”是在同一步完成的。每个候选模式的时间、setup、中心区等待、能耗和负载变化，仍按前文候选决策评价方法计算，此处不再重复展开。

因此，三臂模式选择的本质不是简单增加一只机械臂，而是让每个双臂任务多出“选择哪两只机械臂”的组合决策。例如，某个组合当前 setup 较短，可能使该任务很快完成；但如果它占用了后续单臂任务急需的机械臂，负载差和后续容量下界会变差。另一个组合当前稍慢，却可能释放一只关键机械臂，使后续任务更均衡地展开。自编算法正是通过对每个候选模式进行上述显式评价，在搜索过程中比较这些取舍。

这种处理方式与 CP-SAT 的内部搜索不同。CP-SAT 将不同机械臂组合表示为若干可选区间，由求解器通过传播和分支自动决定是否选择；自编算法则把机械臂组合作为

候选决策的一部分显式展开、评分和排序。这样做的好处是可解释性更强：当三臂方案优于二臂方案时，通常可以从同步等待减少、中心区等待减少、setup 缩短或负载更均衡等方面看出原因。

2.7 速度–能耗非线性优化

前述调度算法默认给定单臂负载速度和双臂协同负载速度。在此基础上，本文进一步讨论速度参数对时间和能耗的影响。速度越大，任务搬运时间通常越短，有利于降低最大完工时间；但速度提高也会增加单位距离运动能耗，使总能耗上升。因此，速度选择本质上是时间目标和能耗目标之间的权衡。

在实现上，速度优化并不直接嵌入每一个调度节点，否则会使搜索空间同时包含离散排序决策和连续速度决策，求解难度明显增加。本文采用外层枚举速度、内层调用调度算法的方式：先给定一组候选速度参数，重新计算各任务处理时间和能耗，再运行调度算法得到对应的 $C_{\max}$ 、 E_{tot} 和 B 。最后比较不同速度下的调度结果，得到速度–能耗非线性关系。

若需要在两类目标之间形成单一评价指标，可使用加权得分

$$F(v_s, v_d) = \lambda \frac{C_{\max}(v_s, v_d)}{C_{\max}^0} + (1 - \lambda) \frac{E_{\text{tot}}(v_s, v_d)}{E_{\text{tot}}^0},$$

其中 v_s 和 v_d 分别表示单臂负载速度和双臂协同负载速度， $C_{\max}^0$ 、 E_{tot}^0 为基准速度下的结果， $\lambda \in [0, 1]$ 表示对时间目标的重视程度。 λ 越大，算法越偏向选择较快速度； λ 越小，算法越偏向节能。

该速度分析与调度算法的关系是外层参数分析，而不是替代调度模型。它可以用于回答两个问题：一是单纯加快机械臂是否一定值得；二是在考虑能耗后，双臂协同速度应取多大才更合理。通过这种方式，本文将离散调度决策和连续速度参数分开处理，使模型解释和实验分析都更加清楚。

参考文献